Drishti : AI-based Smart Surveillance System

(Intelligent Surveillance System for Real-time Headcount and Recognition)

A Major Project Synopsis Submitted to

Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal
Towards Partial Fulfillment for the Award of
Bachelor of Technology
(Computer Science and Engineering)

Under the Supervision of:
Prof. Akshay Dubey

Submitted By:
Riya Kaushal(0827CS221230)
Reeti Shrimal (0827CS221223)
Priyal Waghmare(0827CS221200)
Mandisha Mirza(0827CS233D14)
Priyanka Chirote(0827CS221201)

Department of Computer Science and Engineering
Acropolis Institute of Technology and Research

July - Dec 2025

1. Introduction :

Surveillance is a critical component of modern security infrastructure, yet traditional CCTV systems merely record footage and require constant human supervision for monitoring. This approach is often inefficient, prone to human error, and lacks proactive alerting mechanisms. With the advent of artificial intelligence (AI) and computer vision, surveillance systems can now perform real-time analysis, detect anomalies, and provide actionable insights.

Drishti is an AI-powered smart surveillance system designed to enhance safety and monitoring efficiency. Utilizing **YOLOv8** for real-time person detection and counting, dynamically assesses crowd density. **DeepFace** is integrated for facial recognition, enabling identification of known individuals while labeling unrecognized persons as "Unknown," maintaining privacy and data integrity. The system provides a **live dashboard** displaying headcount, alerts, and recognition results, offering a comprehensive intelligent monitoring solution.

2. Objective :

The proposed project aims to achieve the following objectives:

1. To detect and count individuals in real time using **YOLOv8**.
2. To integrate **DeepFace** for facial recognition, displaying details of recognized individuals.
3. To ensure **data privacy** by labeling unrecognized individuals as "Unknown" without storing their information.
4. To build a **secure and scalable database** for storing recognized individuals' details.
5. To design a **dashboard interface** that displays live video, headcount, alerts, and recognition results.
6. To establish a foundation for **scalability and future extension** and large-scale deployment.

3. Scope :

The scope of *Drishti* includes the design, development, and deployment of an intelligent AI-based surveillance system capable of monitoring, analyzing, and recognizing individuals in real time.

Current Scope:

- ➢ Real-time person detection and headcount using YOLOv8.

- ➢ Facial recognition of known individuals via DeepFace.
- ➢ Secure database storage of recognized individuals' information.
- ➢ Privacy-aware labeling of unrecognized individuals as "Unknown."
- ➢ Dashboard interface for live monitoring, headcount, and recognition results.

Future Scope:

- ➢ Cloud-based storage and centralized dashboards.
- ➢ Mobile app integration for instant notifications to security personnel.
- ➢ Deployment in smart city applications, airports, railway stations, and public venues.

Boundaries and Limitations:

1. Boundaries:

- ➢ Applicable to small to medium-scale areas (offices, classrooms, labs).
- ➢ Works with a single camera; multi-camera support is not included.
- ➢ Facial recognition limited to a pre-built dataset; unknown faces are not stored.

2. Limitations:

- ➢ Accuracy affected by lighting, camera quality, and crowd density.
- ➢ High computational resources required for real-time processing.

4. Study of Existing System :

S. No.	Project Title	Key Features / Description	Techniques Used	Domain/Application
1	Real-Time Human Detection and Tracking using YOLO and DeepSORT	Detects and tracks individuals in CCTV streams	YOLO, DeepSORT	Crowd monitoring, surveillance
2	COVID-19 Social Distancing Monitoring System	Counts people, measures distance for social distancing	YOLOv3, DeepSORT	Pandemic control, public safety

3	Smart Surveillance System using Face Recognition	Recognizes faces from pre-stored dataset	Haar Cascade, LBPH, DeepFace	Campus security, identity verification
4	Automated Attendance System using Face Recognition	Captures faces and marks attendance automatically	CNN, DeepFace	Schools, workplaces
5	Crowd Density Estimation using Computer Vision	Counts individuals to estimate density	Object detection, CNN	Event safety, public gatherings

5. Project Description :

Drishti is an **AI-powered smart surveillance system** designed to provide real-time monitoring, crowd management, and facial recognition in small to medium-scale environments such as offices, classrooms, labs, and public spaces. and identify known individuals via facial recognition — all through a **web-based, real-time dashboard**.

The system combines **YOLOv8 for person detection**, **DeepFace for facial recognition**, to provide fast, accurate, and privacy-conscious surveillance. Built as a responsive web application, Drishti is accessible on any modern browser with a webcam, and the architecture is scalable for future expansion to multiple cameras, cloud integration, and automated notification systems.

Flow of Information :

- **Video Capture** – Live video feed is captured through a webcam.
- **Person Detection** – YOLOv8 detects individuals in real time.
- **Face Recognition** – DeepFace compares detected faces with a pre-built dataset. Recognized persons' details are displayed; unknown individuals are labeled as "Unknown."
- **Database Management** – Details of recognized individuals are securely stored for audits and reference.
- **Dashboard Visualization** – Headcount, zone alerts, and recognition results are displayed live for operators.

Description

The operation begins when the system accesses the live webcam feed. YOLOv8 continuously detects individuals in the frame, Simultaneously, the **DeepFace module** extracts facial embeddings of detected individuals and compares them with a pre-built dataset of known persons. Recognized individuals' names and details are displayed, while unknown faces are labeled "Unknown," ensuring **privacy protection**. All recognized identities are securely stored in a database for audit and record-keeping purposes.

The system interface is **minimal and user-friendly**, optimized for real-time visualization rather than complex interactions. It allows security personnel to monitor multiple parameters — headcount, zone status, alerts, and recognized identities — in a single view. While currently web-based, the system architecture is **scalable** for multi-camera setups, cloud storage, and automated notifications in future iterations.

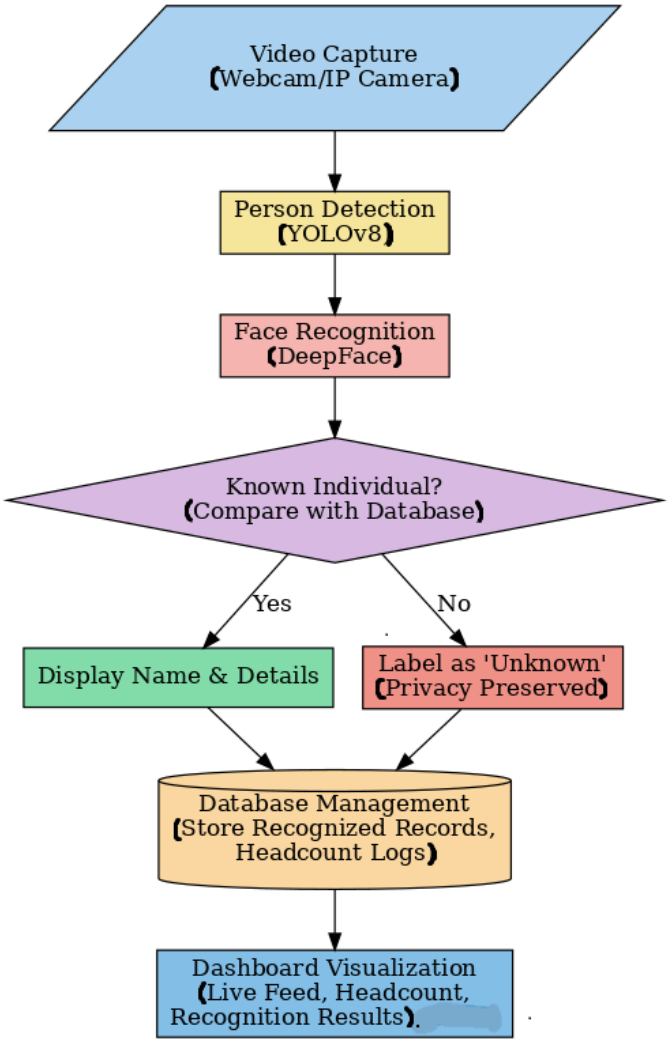


Fig.1 Flow chart

6. Methodology/Planning of the Project work :

The methodology for *Drishti: An AI-based Smart Surveillance System* is designed to ensure real-time person detection, crowd monitoring, facial recognition, and full-stack deployment. The project follows a structured approach divided into several phases:.

1. Requirement Analysis

- Identify key features: real-time person detection, facial recognition, database storage, and frontend dashboard.
- Determine hardware/software needs: webcam, high-performance CPU/GPU, Python, YOLOv8, DeepFace, OpenCV, Flask/Django or FastAPI, MongoDB/MySQL, HTML/CSS/JS or React.js.
- Define constraints such as system scalability, privacy compliance, and real-time processing requirements.

2. Dataset Preparation

- Collect images of known individuals for the facial recognition dataset.
- Preprocess images (resize, normalize) for compatibility with DeepFace.

3. AI Model Implementation

- **Person Detection:** Implement YOLOv8 for fast, real-time detection of individuals in video frames.
- **Facial Recognition:** Integrate DeepFace to identify known individuals and display their details; unknown faces are labeled "Unknown."

4. Backend Development

- Create backend server (Flask/Django/FastAPI) to handle:
  - Video stream processing.
  - Integration of YOLOv8 and DeepFace modules.
  - Database management for storing recognized identities and headcount logs.
  - API endpoints for real-time communication with frontend.

5. Frontend Development

- Build an interactive dashboard using HTML/CSS/JS or React.js to display:
  - Live video feed from the webcam.
  - Real-time headcount.
  - Recognized individuals and their details.

    - Historical statistics and records from the database.
- ➢ Implement real-time updates using **WebSockets** or AJAX calls to fetch processed data from the backend.

6. System Integration

- ➢ Connect frontend and backend:
    - Webcam captures video → backend processes YOLOv8 detection and DeepFace recognition.
    - Processed data (headcount, alerts, recognized faces) → sent via API/WebSocket → frontend dashboard.
    - Dashboard updates dynamically, reflecting real-time status.

7. Testing and Validation

- ➢ Unit testing for each module: person detection, facial recognition
- ➢ Integration testing to ensure smooth communication between frontend, backend, and database.
- ➢ Validate system under different conditions: lighting variations, crowd densities, and camera angles.

8. Deployment and Monitoring

- ➢ Deploy the full-stack system on a local server or cloud platform.
- ➢ Monitor real-time performance and optimize for low-latency processing.
- ➢ Plan for **future expansion**: multi-camera support, cloud storage, and mobile notifications.

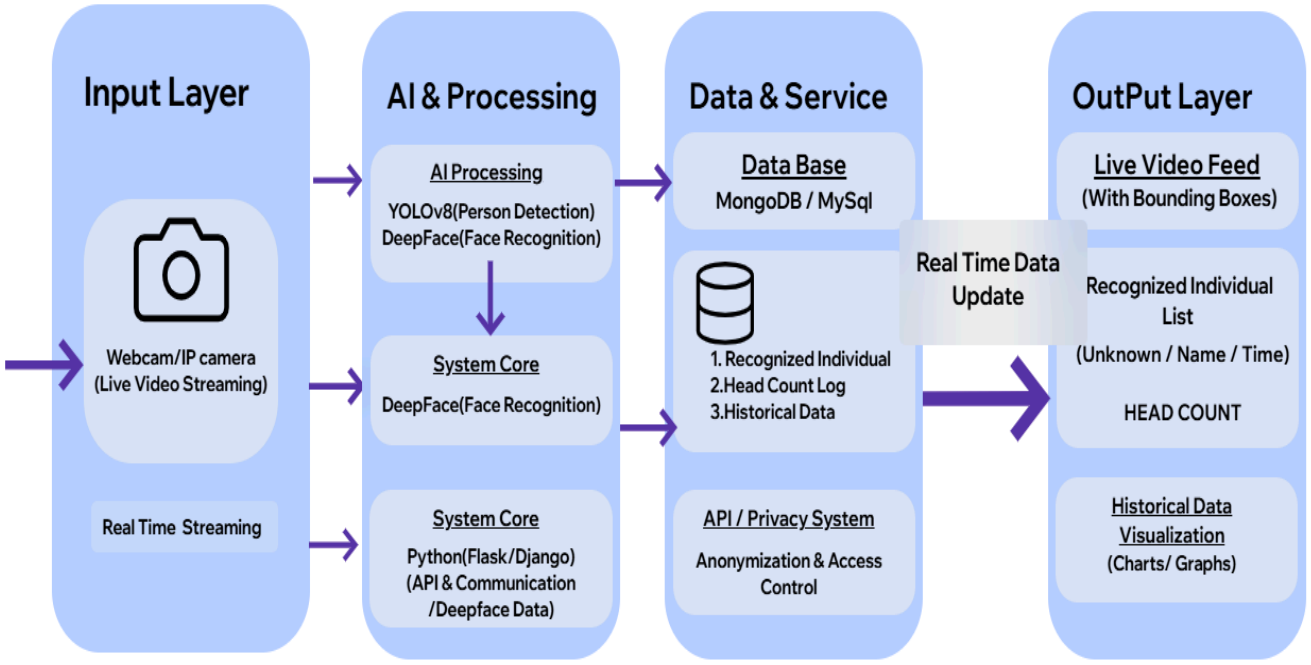
2.Gantt Chart :

S. NO.	TASK TITLE	WEEK 1	WEEK 2	WEEK 4	WEEK 5	WEEK 6	WEEK 7	WEEK 8	WEEK 9	WEEK 10	WEEK 11
1	Requirement Analysis										
2	Data Collection/ Preprocessing										
3	Model Development										
4	Front-end Development										
5	Integration and Testing										
6	Deployment and Validation										

7. Features:

1. **Real-Time Person Detection:** Detects individuals in the camera's field of view using YOLOv8.
2. **Automatic Headcount:** Counts the number of people in the monitored area in real time.
3. **Facial Recognition:** Identifies known individuals using DeepFace and a pre-built dataset.
4. **Privacy-Aware Monitoring:** Unknown individuals are labeled "Unknown" and not stored.
5. **Interactive Dashboard:** Web-based interface showing live video feed, headcount, alerts, and recognized persons.
6. **Frontend-Backend Integration:** Backend processes AI computations; frontend displays real-time results via APIs/WebSockets.
7. **Database Management:** Stores records of recognized individuals, headcount, and zone data securely.
8. **Scalable Architecture:** Modular design ready for multi-camera setups, cloud storage, and future enhancements.

8. System Architecture:



The system follows a modular and layered architecture with clear interaction between components. Here's a brief explanation of how it works:

1. Input Layer

- ➢ Captures live video feed from a webcam or IP camera via the web application.
- ➢ Supports real-time monitoring of multiple individuals for crowd analysis and facial recognition.

2. Front-End (Client-Side)

- ➢ Built using HTML5, CSS (Tailwind), and JavaScript/React.js for a responsive, interactive UI.
- ➢ Dashboard shows live video feed, headcount, and recognized individuals.
- ➢ User-friendly layout with clear visual alerts and minimal text for quick monitoring.

3. Back-End (Server-Side)

- ➢ Developed using Python (Flask/Django/FastAPI) to handle video processing, AI computations, and API requests.
- ➢ Integrates YOLOv8 for person detection,and DeepFace for facial recognition.
- ➢ Manages database operations and real-time communication with frontend via APIs or WebSockets.

4. AI Processing Module

- ➢ **Person Detection (YOLOv8):** Detects individuals in each frame for headcount and spatial analysis.
- ➢ **Facial Recognition (DeepFace):** Matches detected faces with a pre-built dataset and displays recognized individuals.

5. Database & Storage

- ➢ Stores records of recognized individuals, timestamps, headcount logs, information.
- ➢ Maintains historical data for analysis and reporting.
- ➢ Ensures secure storage with encryption and access controls.

6. Security & Privacy Module

- ➢ Labels unknown individuals as “Unknown” without storing their data.
- ➢ Secures communication between frontend and backend using HTTPS/SSL.
- ➢ Optional user authentication for restricted access to dashboard and database.

9. User Interface (UI) :

The UI of Drishti is designed to be intuitive, responsive, and informative, allowing security personnel or operators to monitor real-time video, crowd status, and recognized individuals efficiently. The focus is on clarity, accessibility, and minimal interaction.

1. Dashboard Layout

- **Live Video Feed Panel:** Shows the real-time camera feed in the largest section of the screen.
- **Headcount Display:** Shows the total number of people detected in real time.
- **Recognized Individuals Panel:** Lists names and details of recognized persons from the facial recognition database. Unknown individuals are labeled as "Unknown."

2. Navigation & Interaction

- Minimal buttons for switching cameras, starting/stopping monitoring, and accessing historical logs.
- **Responsive Design:** Dashboard adapts to desktops, laptops, and large-screen tablets.
- Optional hover or click-based details for zone statistics or recognized individual history.

3. Visual Design & Accessibility

- High-contrast color
- Large fonts and icons for quick visibility of critical information.
- Clean layout without clutter to ensure focus on **live monitoring and alerts**.

4. Real-Time Updates

- Frontend communicates with the backend via **APIs or WebSockets**, updating:
  - Live headcount
  - Recognized individual information
- Updates occur **without page refresh**, ensuring smooth and continuous monitoring.

10. Technology Stack:

Drishti is designed using a full-stack approach, combining AI, frontend, backend, and database technologies to provide a real-time, scalable, and efficient surveillance system.

1. Hardware Components

- ➢ Webcam / IP Camera: Captures live video feed for person detection and facial recognition.
- ➢ High-Performance CPU/GPU: Required for real-time AI processing and video analysis.
- ➢ Server / Local Machine: Hosts backend services and database.

2. Frontend Technologies (Client-Side)

- ➢ HTML5, CSS3, Tailwind CSS: For responsive and accessible layout design.
- ➢ JavaScript / React.js: For interactive dashboard, real-time updates, and dynamic UI components.
- ➢ WebSockets / AJAX: For seamless real-time communication with the backend.

3. Backend Technologies (Server-Side)

- ➢ Python (Flask / Django / FastAPI): For handling API requests, AI model processing, and server-side logic.
- ➢ OpenCV: For video processing and frame extraction.
- ➢ YOLOv8: For real-time person detection.
- ➢ DeepFace: For facial recognition and matching with the pre-built dataset.

4. Database & Storage

- ➢ MongoDB / MySQL / PostgreSQL: For storing recognized individual records, headcount logs, and zone data.
- ➢ Cloud Storage (Optional): For hosting AI models and maintaining scalable data storage.

5. AI & Machine Learning Tools

- ➢ YOLOv8: State-of-the-art object detection for accurate person detection.
- ➢ DeepFace: Pre-trained facial recognition framework for identifying known individuals.

6. Security & Privacy

- ➢ SSL / HTTPS: Secure communication between frontend and backend.
- ➢ Privacy Modules: Ensures unknown individuals are labeled “Unknown” without storing sensitive data.
- ➢ Access Controls: Optional authentication for restricted dashboard access.

7. Development & Deployment Tools

- ➢ VS Code : IDEs for code development.
- ➢ Git / GitHub: Version control for project code.

11. Testing Plan :

1. **Unit Testing:** Verify individual modules (YOLOv8 detection, DeepFace recognition, database operations, API communication) using sample inputs and edge cases.

2. **Integration Testing:** Ensure seamless interaction between modules—video feed → detection → recognition → dashboard.

3. **Performance Testing:** Check real-time processing speed (FPS), AI inference time, and CPU/GPU usage under varying crowd densities.

4. **Functional Testing:** Validate core features—headcount, facial recognition, dashboard display, and database logging.
5. **Security & Privacy Testing:** Ensure unknown individuals are not stored, data transmission is secure (SSL/HTTPS), and dashboard access is restricted.

6. **User Acceptance Testing (UAT):** Collect user feedback on usability, alert clarity, and system responsiveness; make adjustments if needed.

7. **Regression Testing:** Re-run tests after updates to confirm existing functionality is intact.

12. Expected Outcome :

Expected Outcomes :

- ➢ Accurate detection and counting of individuals in real time.
- ➢ Recognition of known individuals and labeling of unknowns.
- ➢ Privacy protection by not storing data of unrecognized individuals.
- ➢ Real-time dashboard displaying video feed, headcount, and recognized individuals.
- ➢ Historical logging of recognized individuals, headcount.
- ➢ Smooth, real-time processing with minimal latency.

Benefits to Society :

- **Enhanced Security:** Helps monitor public and private spaces in real time, reducing risk of overcrowding and unauthorized access.
- **Privacy-Conscious Surveillance:** Recognizes individuals without storing unnecessary personal data, protecting privacy.
- **Support for Security Personnel:** Reduces manual monitoring effort, enabling faster and more informed decisions.
- **Applicability in Multiple Sectors:** Useful in offices, campuses, labs, small events, and restricted areas.
- **Data-Driven Insights:** Historical logs help authorities plan better for crowd management and safety measures.
- **Foundation for Smart Cities:** Lays groundwork for AI-driven, automated surveillance systems in urban planning.

13. Resources and Limitations :

Resources Required

- Hardware: Webcam or IP camera, computer/laptop with sufficient CPU/GPU for real-time processing.
- Software: Python, OpenCV, YOLOv8, DeepFace, Flask/Django/FastAPI for backend, HTML5/CSS/JS (or React) for frontend.
- Database: MongoDB, MySQL, or PostgreSQL for storing recognized individuals and logs.
- Development Tools: IDE (VS Code), Git/GitHub for version control.
- Optional: Cloud storage for scalability and multi-camera integration.

Limitations

- Limited coverage area when using a single webcam; not suitable for very large spaces.
- Performance depends on lighting conditions, camera angle, and resolution.
- Recognition accuracy relies on the quality and size of the pre-built face dataset.
- High computational requirements for real-time detection and recognition with multiple cameras.
- Currently designed for small to medium-scale surveillance; may need optimization for large-scale deployment.

14. Conclusion

Drishti is an AI-based smart surveillance system that integrates real-time person detection, automatic headcount, and facial recognition into a single platform. By leveraging YOLOv8 for accurate detection and DeepFace for recognizing known individuals, it provides reliable monitoring while maintaining privacy for unknown persons. The system's interactive dashboard enables security personnel to view live feeds, monitor crowd density, and access recognized individual details efficiently.

Designed with a modular and scalable architecture, Drishti can be expanded to include multiple cameras, cloud storage, and advanced AI features in the future. Its automated alerts and real-time analytics help prevent overcrowding, enhance security, and reduce the need for manual monitoring. Overall, Drishti addresses the limitations of traditional surveillance systems by providing an intelligent, secure, and efficient solution for small to medium-scale environments such as offices, campuses, labs, and public gatherings, while laying the groundwork for future applications in smart city surveillance.

15. References

[1]. Deep Face Recognition: A Survey Mei Wang, Weihong Deng (2018) A comprehensive survey of face recognition methods (architectures, losses, datasets). Good for background/literature review.

[2]. The Elements of End-to-end Deep Face Recognition: A Survey of Recent Advances Hang Du, Hailin Shi, Dan Zeng, Xiao-Ping Zhang, Tao Mei (2020) Focuses on full pipeline: face detection → alignment → representation.

[3]. A Survey on Deep Learning Based Face Recognition (ScienceDirect, 2019) ￼Another survey emphasizing deep-learning face recognition,

[4]. Monitoring COVID-19 Social Distancing with Person Detection and Tracking via fine-tuned YOLO v3 and Deepsort Punn, Sonbhadra, Agarwal, Rai (2020)

[5]. A Lightweight Person Detector for Surveillance Footage Based on YOLOv8n Wang, Q.; Feng, G.; Li Z. (2025) ￼ Very recent. Focused on modifying YOLOv8n for efficient person detection in surveillance video.

[6]. Deep-learned faces: a survey Samadhi P. K. Wickrama Arachchilage & Ebroul Izquierdo(2020) Covers the challenges of face recognition "in the wild" (pose, lighting, expression).

[7]. YOLO Based Real Time Human Detection Using Deep Learning Y. M. Jaswanth Kumar, P Valarmathi (2023) More directly relevant to the human detection component. Helps with baselines for detecting people with YOLO.